

The background of the entire page is composed of several overlapping, wavy, organic shapes in various shades of green, ranging from a deep forest green to a lighter, vibrant green. These shapes create a sense of movement and depth, framing the central text.

NISHAN

**SOFTWARE REQUIREMENTS
SPECIFICATION DOCUMENT**

Table of Contents

1. Introduction	4
1.1 Purpose	5
1.2 Document Conventions	5
1.3 Project Scope	6
2. Overall Description	7
2.1 Product Perspective	7
2.2 User Classes and Characteristics	8
2.3 Operating Environment	9
2.4 Design and Implementation Constraints	10
2.5 Assumptions and Dependencies	11
2.5.1 Assumptions	11
2.5.2 Dependencies	11
3. System Features	12
3.1 Multimodal Input System (MIS-01)	12
3.2 AI-Based Location Understanding (AILU-02)	13
3.3 Geospatial Search and Location Matching (GSLM-03)	14
3.4 Interactive Map and Location Visualization (IMLV-04)	14
3.5 Location Result Details (LRD-05)	15
3.6 Error Handling and Result Feedback (EHRF-06)	16
3.7 Fully Dressed Use Cases	16
3.8 Feature-to-Requirement Summary	24
4. Data Requirements	24
4.1 Logical Data Model	25
4.2 Data Dictionary	26
4.3 Reports	29
4.4 Data Validation and Integrity	30
4.5 Data Security and Privacy	30
4.6 Data Lifecycle	31
5. External Interface Requirements	31
5.1 User Interfaces	31
5.2 Software Interfaces	35
5.3 Hardware Interfaces	38

5.4 Communication Interfaces	38
5.5 Interface Error Handling	39
5.6 Interface Security Requirements	40
5.7 Interface Summary	40
6. Quality Attributes	41
6.1 Usability.....	41
6.2 Performance	41
6.3 Security	42
6.4 Reliability	42
6.5 Availability.....	43
6.6 Scalability.....	43
6.7 Maintainability	43
6.8 Compatibility	44
6.9 Data Integrity	44
6.10 AI Response Quality	45
6.11 Responsiveness	45
6.12 Deployment and Portability	45
6.13 Quality Attribute Summary	46
7. Internationalization and Localization.....	46
7.1 Language Support.....	46
7.2 Pakistani Geographic Context	47
7.3 Local Place Names.....	48
7.4 Geographic Conventions	48
7.5 Date and Time.....	48
7.6 User Interface Localization	49
7.7 AI Language Processing	49
7.8 Localization Requirements Summary	50
8. Other Requirements	50
8.1 Legal and Regulatory Requirements.....	50
8.2 Third-Party Service Dependencies	51
8.3 API and Usage Limitations	51
8.4 Data Retention	51

8.5 Backup and Recovery	52
8.6 Deployment Requirements	52
8.7 Environment Configuration	53
8.8 OpenStreetMap Requirements	53
8.9 AI Usage Requirements	53
8.10 Technology Constraints	54
8.11 Open Source and Third-Party Components	55
8.12 Project Constraints	55
8.13 Future Extensibility	55
8.14 Other Requirements Summary	56
9. Appendices	56
9.1 Appendix A: Glossary	56
9.2 Appendix B: Acronyms and Abbreviations	58
9.3 Appendix C: Assumptions	58
9.4 Appendix D: Constraints	59
9.5 Appendix E: Technology Stack	59
9.6 Appendix F: Core System Flow	60
9.7 Appendix G: Release 1.0 Scope	60
9.8 Appendix H: Requirement Traceability	61

1. Introduction

Nishaan: An AI-Powered Geospatial Identification and Location Assistance System is a web-based platform designed to help users identify and locate real-world places and landmarks through natural and intuitive forms of interaction. The system allows users to describe a location-related situation using text, voice, or images, reducing the need for users to know the exact name, address, or coordinates of a place.

Nishaan uses artificial intelligence to interpret the information provided by the user and extract meaningful location-related details from it. The system then combines this understanding with geographic data and spatial search capabilities to identify potentially relevant locations. Geographic information is managed using PostgreSQL with PostGIS, while OpenStreetMap provides real-world geographic and place data. Identified locations are presented through an interactive map using Leaflet, allowing users to visually understand where the relevant places are located.

The system follows a modern web-based architecture consisting of a Next.js and Tailwind CSS frontend, a FastAPI and Python backend, a Groq API-powered AI layer, and a PostgreSQL/PostGIS database. The application is intended to provide a simple and accessible interface while combining natural-language understanding with geographic search and visualization.

The core objective of Nishaan is to make location discovery more accessible by allowing users to communicate their needs naturally rather than relying entirely on exact place names, addresses, or coordinates. The system is particularly focused on situations where a user may recognize or describe a place based on surrounding landmarks, visual characteristics, or contextual information but may not know its precise location.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the functional and non-functional requirements of the **Nishaan** system. It describes the expected behavior, features, interfaces, data requirements, system constraints, quality attributes, and interactions between users and the system.

This document serves as a reference for the development team, system designers, testers, project managers, stakeholders, and other individuals involved in the development and evaluation of Nishaan. It provides a common understanding of what the system is expected to accomplish and establishes the requirements against which the completed system can be evaluated.

The SRS covers the requirements for **Release 1.0** of Nishaan, including user interaction through text, voice, and image-based input, AI-powered interpretation through the Groq API, geographic and spatial search using PostgreSQL/PostGIS, geographic data integration through OpenStreetMap, and location visualization through Leaflet.

The document will also define the system's external interfaces, database requirements, security and performance expectations, deployment requirements, and other constraints relevant to the implementation of the initial release.

1.2 Document Conventions

The conventions used in this Software Requirements Specification (SRS) are outlined below.

a) Text Styles and Formatting

- **Bold text** is used for section headings, subsection titles, important terms, technologies, and system components.
- *Italicized text* may be used for notes, examples, or explanatory information.
- Numbered headings are used to maintain a structured hierarchy throughout the document.

b) Requirement Numbering

All requirements will be uniquely identified using a structured numbering format:

- **FR#**: Represents a Functional Requirement. For example, FR1, FR2, FR3.
- **NFR#**: Represents a Non-Functional Requirement. For example, NFR1, NFR2, NFR3.
- **UIR#**: Represents a User Interface Requirement. For example, UIR1, UIR2.
- **DR#**: Represents a Data Requirement where applicable.

c) Figures and Tables

- All diagrams, including use case diagrams, class diagrams, architecture diagrams, and ER diagrams, will be labeled as **Figure X.Y**.
- All tables will be labeled as **Table X.Y** and will have descriptive titles.
- References to figures and tables within the document will use formats such as “**see Figure 3.1**” and “**refer to Table 4.1**.”

This follows the convention used in the previous SRS, where figures and tables are numbered according to their respective sections.

d) Language and Terminology

The document uses clear and unambiguous English to describe system behavior and requirements. Technical terms related to artificial intelligence, geospatial data, spatial databases, mapping, and web development will be used consistently throughout the document.

Important domain-specific terms such as **AI processing, spatial search, geographic coordinates, PostGIS, OpenStreetMap, geospatial data, landmark, and location identification** will be defined in the glossary in **Appendix A**.

1.3 Project Scope

Nishaan is an AI-powered geospatial assistance platform designed to simplify the process of identifying and locating real-world places. Traditional location searches often require users to know an exact place name, address, or other precise information. Nishaan provides an alternative approach by allowing users to communicate what they know about a location through **text descriptions, spoken descriptions, or images**.

The system processes the user's input using an AI layer powered by the **Groq API**. The AI component interprets the provided information and extracts relevant contextual or location-related details. These details are then used by the backend to perform geographic and spatial searches using **PostgreSQL and PostGIS** and geographic information obtained through **OpenStreetMap**.

The resulting locations are displayed to the user through an interactive map powered by **Leaflet**. This allows users to examine potential locations visually and understand their geographic relationship to surrounding places and landmarks.

From a user and system perspective, the software is designed to:

- Allow users to provide location-related information through **text, voice, or images**.
- Interpret user-provided information using AI-powered processing.
- Identify relevant geographic or landmark information from the provided input.
- Perform spatial searches using geographic coordinates and PostGIS.
- Retrieve and use real-world geographic information from OpenStreetMap.
- Present potential locations through an interactive map.
- Provide users with an intuitive method of discovering locations without requiring exact place names or coordinates.

Release 1.0

The initial release of Nishaan will include the following major capabilities:

- i. Text-based location description and input
- ii. Voice-based location description and input
- iii. Image-based location input and analysis
- iv. AI-powered interpretation using the Groq API
- v. Geographic and spatial search using PostgreSQL and PostGIS
- vi. Integration with OpenStreetMap geographic data
- vii. Interactive location visualization using Leaflet
- viii. Web-based user interface developed using Next.js and Tailwind CSS
- ix. Backend services developed using FastAPI and Python
- x. Cloud-based deployment using Vercel and Render/Railway

This SRS defines the functionality, users, system constraints, data requirements, interfaces, quality attributes, and expected interactions for Nishaan.

2. Overall Description

2.1 Product Perspective

Nishaan is a web-based AI-powered geospatial assistance system that combines artificial intelligence, geographic data, and spatial search to help users identify and locate real-world places based on descriptive information.

The system follows a client-server architecture in which the **Next.js** frontend provides the user interface, while the **FastAPI** backend manages communication between the frontend, AI services, and geographic database. The system uses the **Groq API** to process and interpret user-provided text, voice, and image inputs. The extracted information is then used to perform location-based searches through **PostgreSQL with PostGIS**.

Geographic information required by the system is obtained through **OpenStreetMap**, while **Leaflet** is used to present geographic results through an interactive map interface. The overall relationship between the major system components is illustrated below.

The system is designed as a modular application so that the frontend, backend, AI processing, database, and mapping components have clearly defined responsibilities. This separation allows individual components to be maintained or improved without requiring major changes to the entire system.

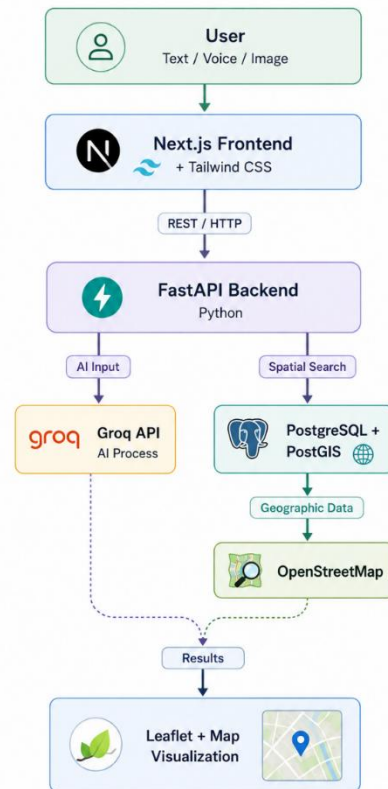


Figure 2.1: Nishaan System Architecture

2.2 User Classes and Characteristics

Nishaan is primarily designed for users who need assistance identifying or locating real-world places but may not know the exact name, address, or coordinates of the location.

2.2.1 General Users

General users are the primary users of Nishaan. They interact with the system through the web interface and provide information about a location using text, voice, or an image.

Their activities may include:

- Entering a textual description of a place or landmark.
- Providing a spoken description through the device microphone.
- Uploading or providing an image related to a location.

- Reviewing the AI interpretation of their input.
- Viewing identified or potentially relevant locations.
- Examining results on an interactive map.
- Using geographic information to understand the location of a place.

Users are not expected to have specialized technical knowledge. The interface should therefore provide simple and understandable methods for submitting information and viewing results.

2.2.2 System Administrators

If an administrative component is included in the final implementation, administrators will be responsible for monitoring and managing system-level information and operations.

Potential administrative responsibilities may include:

- Monitoring system activity.
- Managing geographic or system data where required.
- Reviewing system errors or problematic results.
- Managing configuration-related information.
- Monitoring external service integration.

The exact administrative functionality will be finalized during the requirements analysis and development phase.

2.3 Operating Environment

Nishaan will operate as a web-based application and will be accessible through modern web browsers on supported computing devices.

2.3.1 Client Environment

The system will support:

- Desktop computers and laptops.
- Smartphones.
- Tablets.

The application will be accessed through modern browsers such as Chrome, Edge, Firefox, or other browsers supporting the required web technologies.

The user device may require:

- Microphone access for voice-based input.
- Camera or image upload access for image-based input.
- A stable internet connection for communication with the application backend and external services.

2.3.2 Frontend Environment

The user interface will be developed using:

- Next.js
- React
- Tailwind CSS
- Leaflet

The frontend will communicate with the FastAPI backend through web-based API requests.

2.3.3 Backend Environment

The backend will be developed using:

- Python
- FastAPI

The backend will manage API requests, AI processing workflows, geographic search operations, and communication with the database and external services.

2.3.4 Database Environment

Nishaan will use:

- PostgreSQL as the relational database management system.
- PostGIS as the spatial database extension.

PostGIS will allow the system to store and process geographic coordinates and perform spatial searches required by the application.

2.3.5 External Services

The system will depend on external services including:

- Groq API for AI-powered processing.
- OpenStreetMap for geographic and mapping data.

2.3.6 Deployment Environment

The frontend will be deployed using Vercel, while the backend and supporting services may be deployed using Render or Railway, according to the deployment configuration selected for the project.

2.4 Design and Implementation Constraints

The development and operation of Nishaan are subject to the following constraints:

- The frontend must be developed using Next.js and Tailwind CSS.
- The backend must use FastAPI with Python.
- AI-based processing must use the Groq API rather than a locally hosted AI model.
- Geographic and spatial data processing must use PostgreSQL with PostGIS.
- The system must use OpenStreetMap as its geographic data source where applicable.
- Leaflet will be used to display geographic information through an interactive map.
- API credentials and other sensitive configuration values must not be hard-coded into the application source code. They must be maintained using environment variables such as .env files during development and appropriate environment configuration during deployment.
- The quality of AI-generated results may depend on the clarity and completeness of the information provided by the user.
- Voice and image-based functionality depends on the capabilities and permissions of the user's device and browser.
- The availability and usage limits of external services such as the Groq API and geographic data services may affect system functionality.
- An active internet connection is required for communication between the frontend, backend, AI service, database services, and other external resources.

2.5 Assumptions and Dependencies

2.5.1 Assumptions

The system is developed based on the following assumptions:

- ❖ Users will provide meaningful and sufficiently clear information about the location they are trying to identify.
- ❖ Users will have access to a device capable of running a modern web browser.
- ❖ Users requiring voice input will provide the necessary microphone permissions.
- ❖ Users providing images will upload images that contain useful visual or contextual information.
- ❖ Geographic information required for the system will be available through the selected geographic data sources.
- ❖ The AI service will be available when AI-powered interpretation is requested.
- ❖ Users will have an active internet connection while using the system.
- ❖ The initial release will primarily focus on location identification and geographic assistance rather than full turn-by-turn navigation.

2.5.2 Dependencies

Nishaan depends on several external and internal components:

Dependency	Purpose
------------	---------

Groq API	AI-powered interpretation of user-provided information
OpenStreetMap	Geographic and place information
PostgreSQL	Structured application and geographic data storage
PostGIS	Spatial data processing and location-based queries
Leaflet	Interactive map visualization
FastAPI	Backend API and system integration
Next.js	Web application frontend
Vercel	Frontend deployment
Render/Railway	Backend/service deployment

The availability, API limitations, service changes, and connectivity of these external dependencies may affect the operation of Nishaan.

3. System Features

Nishaan consists of a set of integrated features that allow users to provide location-related information through multiple input methods, process that information using artificial intelligence, identify relevant geographic locations, and visualize the results on an interactive map.

The major system features of Nishaan are:

1. Multimodal Input System (MIS-01)
2. AI-Based Location Understanding (AILU-02)
3. Geospatial Search and Location Matching (GSLM-03)
4. Interactive Map and Location Visualization (IMLV-04)
5. Location Result Details (LRD-05)
6. Error Handling and Result Feedback (EHRF-06)

3.1 Multimodal Input System (MIS-01)

3.1.1 Description

The Multimodal Input System allows users to provide information about a location using different forms of input. Instead of requiring an exact place name or address, users can describe what they know about a location through **text, voice, or an image**.

The feature acts as the primary entry point into Nishaan and prepares the submitted information for AI-based processing.

Priority: High

3.1.2 Functional Requirements

- **FR1:** The system shall allow users to enter a textual description of a location or landmark.
- **FR2:** The system shall allow users to provide a spoken description of a location in **Urdu** using a supported microphone.
- **FR2.1:** The system shall process Urdu voice input and extract location-related information from the spoken description.
- **FR3:** The system shall allow users to upload an image related to the location they want to identify.
- **FR4:** The system shall validate submitted input before sending it for processing.
- **FR5:** The system shall allow the user to submit the provided input for analysis.
- **FR6:** The system shall display an appropriate message when the submitted input is missing, invalid, or unsupported.
- **FR7:** The system shall transfer valid user input to the backend for further processing.

3.2 AI-Based Location Understanding (AILU-02)

3.2.1 Description

This feature uses the **Groq API** to process information submitted by the user and identify meaningful location-related characteristics.

The AI component interprets textual descriptions, spoken information, and image-based information and extracts useful contextual details that can be used by the geographic search component.

The AI layer is responsible for understanding the user's description, while geographic matching is handled separately by the spatial search system.

Priority: High

3.2.2 Functional Requirements

- **FR8:** The system shall send valid user input to the AI processing service through the backend.
- **FR9:** The system shall process textual descriptions using the Groq API.
- **FR10:** The system shall process supported voice-based input and extract relevant location information.
- **FR11:** The system shall process supported image input and extract relevant visual or contextual information.
- **FR12:** The system shall identify location-related entities, landmarks, descriptions, or contextual clues from the submitted input.
- **FR13:** The system shall convert AI-processed information into structured data that can be used by the geographic search component.
- **FR14:** The system shall handle cases where the AI cannot determine sufficient location-related information.

- **FR15:** The system shall not treat AI-generated information as a confirmed geographic location until it has been matched against available geographic data.

3.3 Geospatial Search and Location Matching (GSLM-03)

3.3.1 Description

The Geospatial Search and Location Matching feature uses the information extracted by the AI component to identify potentially relevant real-world locations.

The system uses PostgreSQL with PostGIS to store and process geographic information and perform spatial queries. Geographic information from OpenStreetMap is used where applicable to support the identification and matching of places.

This feature forms the bridge between the AI's understanding of the user's description and the actual geographic locations presented by Nishaan.

Priority: High

3.3.2 Functional Requirements

- **FR16:** The system shall receive structured location-related information from the AI processing component.
- **FR17:** The system shall search available geographic data for locations matching the extracted information.
- **FR18:** The system shall use PostGIS to perform supported spatial searches involving geographic coordinates.
- **FR19:** The system shall use geographic information obtained from OpenStreetMap where applicable.
- **FR20:** The system shall identify potentially relevant locations based on available geographic and contextual information.
- **FR21:** The system shall support proximity-based geographic searches where sufficient coordinates are available.
- **FR22:** The system shall return one or more potential locations when matching results are available.
- **FR23:** The system shall indicate when no sufficiently relevant geographic result can be found.
- **FR24:** The system shall pass matched geographic coordinates and relevant location information to the map visualization component.

3.4 Interactive Map and Location Visualization (IMLV-04)

3.4.1 Description

This feature presents geographic search results through an interactive map using **Leaflet** and geographic map data from **OpenStreetMap**.

The map allows users to visually examine potential locations rather than relying solely on textual results.

Priority: High

3.4.2 Functional Requirements

- **FR25:** The system shall display a map after geographic search results are available.
- **FR26:** The system shall display identified or potentially relevant locations as markers on the map.
- **FR27:** The system shall associate each displayed marker with its corresponding geographic coordinates.
- **FR28:** The system shall allow users to interact with the displayed map.
- **FR29:** The system shall allow users to zoom and move around the map.
- **FR30:** The system shall display relevant location information when a user selects a map marker.
- **FR31:** The system shall update the map based on the geographic results returned by the backend.
- **FR32:** The system shall clearly distinguish between available potential results where multiple locations are returned.

3.5 Location Result Details (LRD-05)

3.5.1 Description

The Location Result Details feature provides users with additional information about locations identified by the system.

The feature allows users to understand why a location may be relevant and provides geographic information associated with the selected result.

Priority: Medium

3.5.2 Functional Requirements

- **FR33:** The system shall display the name of a matched location when available.
- **FR34:** The system shall display available geographic information associated with a matched location.
- **FR35:** The system shall display the location on the interactive map.
- **FR36:** The system shall allow users to select an individual result when multiple potential locations are available.
- **FR37:** The system shall present available contextual information associated with the selected location.
- **FR38:** The system shall clearly indicate when a displayed result is a potential match rather than a guaranteed identification.

3.6 Error Handling and Result Feedback (EHRF-06)

3.6.1 Description

The Error Handling and Result Feedback feature ensures that Nishaan provides understandable responses when input cannot be processed, external services are unavailable, or no suitable geographic match can be identified.

The system should not leave the user uncertain about whether their request was processed successfully.

Priority: Medium

3.6.2 Functional Requirements

- **FR39:** The system shall display an appropriate message when user input cannot be processed.
- **FR40:** The system shall notify the user when an external service required for processing is unavailable.
- **FR41:** The system shall notify the user when no relevant geographic match is found.
- **FR42:** The system shall prevent invalid requests from being unnecessarily submitted to external services.
- **FR43:** The system shall provide the user with an opportunity to provide revised information after an unsuccessful search.
- **FR44:** The system shall handle backend or API errors without exposing sensitive technical information to the user.

3.7 Fully Dressed Use Cases

The following fully dressed use cases describe the primary interactions between users and the Nishaan system.

3.7.1 UC-01 Submit Location Description

Use-Case Section	Details
Use-Case Name	Submit Location Description
Scope	Nishaan System
Level	User Goal
Primary Actor	General User
Supporting Actors	Nishaan Backend, Groq API
Priority	High
Stakeholders and Interests	User: Wants to describe a location without needing to know its exact name or coordinates. Nishaan System: Needs to receive and process valid user input. AI Service: Processes the submitted information to identify meaningful location-related details.

Preconditions	The Nishaan system is operational. The user has access to the Nishaan web application. The user has entered a valid location-related description.
Success Guarantee	The user's description is successfully submitted and transferred to the AI processing component for analysis.
Main Success Scenario	1. User opens the Nishaan application. 2. User selects the text input option. 3. User enters a description of the location. 4. System validates the submitted information. 5. User submits the description. 6. System sends the request to the FastAPI backend. 7. Backend prepares the input for AI processing. 8. System sends the relevant information to the Groq API. 9. AI processing is initiated. 10. The system continues to the location understanding process.
Extensions	3a. Empty Description: If the user submits an empty description, the system asks the user to provide location-related information. 4a. Invalid Input: If the input does not meet the required format, the system displays an appropriate validation message. 8a. AI Service Unavailable: If the Groq API is unavailable, the system informs the user that the request cannot currently be processed.
Special Requirements	User input must be transmitted securely. Sensitive API credentials must not be exposed to the client. The interface should provide clear feedback while the request is being processed
Technology & Data Variation List	Frontend: Next.js and Tailwind CSS. Backend: FastAPI and Python. AI Processing: Groq API. Communication: REST/HTTP API.
Frequency of Occurrence	Frequently, whenever a user wants to identify or locate a place.
Miscellaneous	Future versions may support additional input formats or improved natural-language processing.

3.7.2 UC-02 Submit Voice Description

Use-Case Section	Details
Use-Case Name	Submit Voice Description
Scope	Nishaan System
Level	User Goal
Primary Actor	General User
Supporting Actors	Browser, FastAPI Backend, Groq API
Priority	High
Stakeholders and Interests	User: Wants to describe a location verbally instead of typing. System: Needs to capture and process the user's spoken information.

	AI Service: Needs usable voice-derived information to identify location clues.
Preconditions	The user has opened Nishaan. The user's device has microphone capability. The browser has permission to access the microphone
Success Guarantee	The spoken description is successfully captured and processed for location understanding.
Main Success Scenario	1. User selects the voice input option. 2. System requests microphone permission if required. 3. User grants microphone access. 4. User speaks a description of the location. 5. System captures the voice input. 6. System transfers the input for processing. 7. Backend processes the voice-based request. 8. Relevant information is provided to the AI processing component. 9. The system extracts location-related information. 10. The extracted information is passed to the geographic search component
Extensions	2a. Microphone Permission Denied: The system informs the user that microphone access is required and provides an alternative input method. 4a. No Speech Detected: The system asks the user to try again. 7a. Voice Processing Failure: The system displays an appropriate error and allows the user to retry or use text input.
Special Requirements	Microphone access must require user permission. Voice data must be handled securely. The system should provide clear recording and processing feedback.
Technology & Data Variation List	Frontend: Next.js and Tailwind CSS. Backend: FastAPI and Python. AI Processing: Groq API. Communication: REST/HTTP API.
Frequency of Occurrence	Frequently, depending on user preference.
Miscellaneous	Voice input provides an alternative for users who prefer speaking over typing.

3.7.3 UC-03 Submit Image for Location Analysis

Use-Case Section	Details
Use-Case Name	Submit Image for Location Analysis
Scope	Nishaan System
Level	User Goal
Primary Actor	General User
Supporting Actors	FastAPI Backend, Groq API

Priority	High
Stakeholders and Interests	User: Wants to identify a location using a photograph or other visual information. System: Needs to receive and process the submitted image. AI Service: Extracts useful visual and contextual information from the image.
Preconditions	The user has access to Nishaan. The user has a supported image file or access to a device camera where supported. The image meets the system's supported input requirements
Success Guarantee	The image is successfully processed and useful location-related information is extracted.
Main Success Scenario	1. User selects the image input option. 2. User uploads or selects an image. 3. System validates the image. 4. System sends the image through the backend for processing. 5. Backend prepares the request for AI analysis. 6. AI service analyzes the image. 7. System extracts relevant visual or contextual clues. 8. Extracted information is passed to the geographic search component. 9. The system searches for potentially matching locations
Extensions	2a. Unsupported Image: System informs the user that the selected image cannot be processed. 3a. Invalid File: System rejects the file and requests another image. 6a. Insufficient Visual Information: System informs the user that the image does not provide enough information and allows another input to be submitted.
Special Requirements	Image uploads must be securely handled. The system should provide appropriate processing feedback. Uploaded images should not be unnecessarily retained.
Technology & Data Variation List	Frontend: Next.js and Tailwind CSS. Backend: FastAPI and Python. AI Processing: Groq API. Communication: REST/HTTP API.
Frequency of Occurrence	Frequently, depending on user preference.
Miscellaneous	Future versions may improve image-based location recognition through enhanced AI processing.

3.7.4 UC-04 Analyze Location Information

Use-Case Section	Details
Use-Case Name	Analyze Location Information
Scope	Nishaan System
Level	System Goal
Primary Actor	Nishaan System
Supporting Actors	Groq API

Priority	High
Stakeholders and Interests	User: Wants the system to understand the information they provided. Nishaan: Needs structured information that can be used for geographic searching. Groq API: Processes the submitted information
Preconditions	Valid user input has been received. The backend is operational. The AI service is accessible.
Success Guarantee	The system obtains structured location-related information suitable for geographic searching.
Main Success Scenario	1. Backend receives the user's input. 2. System determines the input type. 3. System prepares the input for AI processing. 4. Backend sends the request to the Groq API. 5. Groq processes the provided information. 6. System receives the AI response. 7. System extracts relevant location-related information. 8. System structures the extracted information. 9. Structured information is passed to the geographic search component.
Extensions	4a. API Failure: The system records the failure and informs the user that analysis could not be completed. 5a. Insufficient Information: If the AI cannot extract useful location clues, the system requests additional information from the user. 6a. Invalid AI Response: The backend validates the returned information and prevents invalid data from being passed to the spatial search component.
Special Requirements	AI responses must be validated before being used by other system components. API keys must remain private. The system should handle AI service failures gracefully
Technology & Data Variation List	Frontend: Next.js and Tailwind CSS. Backend: FastAPI and Python. AI Processing: Groq API. Communication: REST/HTTP API.
Frequency of Occurrence	Every time a user submits an input requiring AI interpretation.
Miscellaneous	AI-generated information represents an interpretation and should not automatically be treated as a confirmed location.

3.7.5 UC-05 Search and Match Geographic Locations

Use-Case Section	Details
Use-Case Name	Search and Match Geographic Locations
Scope	Nishaan System
Level	System Goal
Primary Actor	Nishaan System
Supporting Actors	PostgreSQL/PostGIS, OpenStreetMap

Priority	High
Stakeholders and Interests	User: Wants the most relevant geographic location based on their description. Nishaan: Needs to identify useful geographic matches. Database: Stores and searches geographic information. OpenStreetMap: Provides geographic/place information where applicable.
Preconditions	AI processing has produced usable location-related information. Geographic data is available. PostgreSQL/PostGIS is operational.
Success Guarantee	The system identifies and returns one or more potentially relevant geographic locations.
Main Success Scenario	1. System receives structured location information. 2. System determines the appropriate geographic search criteria. 3. System queries PostgreSQL/PostGIS. 4. Spatial search is performed where geographic coordinates are available. 5. Relevant geographic records are retrieved. 6. The system evaluates the returned results against available information. 7. Potentially relevant locations are identified. 8. Geographic coordinates and location information are returned to the map component.
Extensions	3a. No Matching Location: The system returns no-match information and informs the user. 4a. Insufficient Geographic Information: The system requests additional information before attempting a more precise search. 5a. Geographic Data Unavailable: The system informs the user that the required location information is temporarily unavailable.
Special Requirements	Spatial searches should be optimized for acceptable response times. Geographic coordinates must be stored and processed correctly. The system must distinguish potential matches from confirmed information.
Technology & Data Variation List	Frontend: Next.js and Tailwind CSS. Backend: FastAPI and Python. AI Processing: Groq API. Communication: REST/HTTP API. Geographic Coordinates
Frequency of Occurrence	Every time a successfully analyzed request requires geographic matching.
Miscellaneous	Future versions may implement more advanced ranking or matching algorithms for candidate locations.

3.7.6 UC-06 Display Location Results on Map

Use-Case Section	Details
------------------	---------

Use-Case Name	Display Location Results on Map
Scope	Nishaan System
Level	User Goal
Primary Actor	General User
Supporting Actors	Leaflet/OpenStreetMap
Priority	High
Stakeholders and Interests	User: Wants to visually understand where the identified locations are. System: Needs to accurately represent geographic results. Mapping Component: Displays geographic data in an interactive format.
Preconditions	Geographic search has returned valid coordinates. The map component is available.
Success Guarantee	The user can view and interact with the geographic results on a map.
Main Success Scenario	1. System receives geographic search results. 2. System loads the map. 3. System obtains the relevant map data. 4. System places markers at the returned coordinates. 5. System adjusts the map view to display the relevant results. 6. User selects a marker. 7. System displays information associated with the selected location. 8. User can zoom, move, and inspect the map.
Extensions	2a. Map Loading Failure: The system informs the user that the map could not be loaded. 4a. Invalid Coordinates: The system ignores invalid geographic coordinates and displays valid results. 5a. Multiple Results: The system displays all valid candidate locations and allows the user to inspect them individually.
Special Requirements	Coordinates must be accurately represented. Map controls should be easy to understand. The interface should remain usable on different screen sizes
Technology & Data Variation List	❖ Leaflet. ❖ OpenStreetMap. ❖ Next.js. ❖ Geographic coordinates.
Frequency of Occurrence	Every time a successful geographic search produces results.
Miscellaneous	Future versions may provide additional map visualization features.

3.7.7 UC-07 View Location Details

Use-Case Section	Details
Use-Case Name	View Location Details
Scope	Nishaan System

Level	User Goal
Primary Actor	General User
Supporting Actors	Nishaan System
Priority	High
Stakeholders and Interests	User: Wants additional information about a potential location. System: Wants to provide understandable geographic information
Preconditions	At least one geographic result is available. The result contains valid location information.
Success Guarantee	The user can view the available details of the selected location.
Main Success Scenario	1. System displays available location results. 2. User selects a result or map marker. 3. System retrieves the associated location information. 4. System displays the location name or available identifying information. 5. System displays relevant geographic information. 6. System highlights the selected location on the map.
Extensions	3a. Missing Location Details: If some details are unavailable, the system displays the information that is available. 4a. Multiple Potential Matches: The user can select another result for comparison.
Special Requirements	Location information must be presented clearly. The system should avoid presenting uncertain information as confirmed facts
Technology & Data Variation List	❖ Next.js. ❖ FastAPI. ❖ PostgreSQL/PostGIS. ❖ Leaflet.
Frequency of Occurrence	Whenever users inspect search results.
Miscellaneous	Additional location metadata may be incorporated in future versions.

3.7.8 UC-08 Handle Unsuccessful Location Search

Use-Case Section	Details
Use-Case Name	Handle Unsuccessful Location Search
Scope	Nishaan System
Level	User Goal
Primary Actor	General User
Supporting Actors	Nishaan System
Priority	High
Stakeholders and Interests	User: Wants to understand why a location could not be identified and try again. System: Needs to clearly communicate unsuccessful searches.
Preconditions	The user has submitted a location-related request.

	AI and/or geographic processing has completed.
Success Guarantee	The user understands that the search was unsuccessful and can provide additional information.
Main Success Scenario	1. User submits location information. 2. System processes the input. 3. System performs geographic searching. 4. No sufficiently relevant result is identified. 5. System informs the user that no suitable match was found. 6. System provides an option to submit additional or revised information. 7. User provides additional information. 8. System begins a new analysis.
Extensions	4a. Partial Match: If potentially relevant but uncertain results exist, the system displays them as potential matches. 5a. External Service Failure: If the search failed because an external service was unavailable, the system informs the user that the issue is technical rather than presenting the result as a genuine no-match.
Special Requirements	Error messages should be understandable to non-technical users. Technical details such as API keys, stack traces, or internal errors must not be exposed.
Technology & Data Variation List	❖ Next.js. ❖ FastAPI. ❖ Groq API. ❖ PostgreSQL/PostGIS. ❖ Leaflet.
Frequency of Occurrence	Occasionally, depending on the quality and availability of user-provided information.
Miscellaneous	Future versions may provide suggestions for improving the user's description based on the reason for an unsuccessful search.

3.8 Feature-to-Requirement Summary

To keep the requirements traceable, the functional requirements are grouped as follows:

Feature	Requirement Range	Priority
Multimodal Input System	FR1–FR7	High
AI-Based Location Understanding	FR8–FR15	High
Geospatial Search and Location Matching	FR16–FR24	High
Interactive Map and Location Visualization	FR25–FR32	High
Location Result Details	FR33–FR38	Medium
Error Handling and Result Feedback	FR39–FR44	Medium

4. Data Requirements

This section defines the data that Nishaan requires to receive, process, store, retrieve, and present location-related information. Since Nishaan combines artificial intelligence with geographic and spatial processing, its data requirements include both conventional application data and geospatial data such as coordinates and location geometry.

The system will use PostgreSQL with PostGIS as its primary data storage and spatial processing environment. The data model is designed to support the flow from a user's submitted input through AI analysis and geographic matching to the final location results presented on the interactive map.

4.1 Logical Data Model

The logical data model of Nishaan represents the relationships between user requests, submitted inputs, AI analysis, geographic locations, and search results.

The major entities identified for Release 1.0 are:

1. Search Request
2. Input
3. AI Analysis
4. Location
5. Search Result

4.1.1 Search Request

The Search Request entity represents a single location-identification request submitted through Nishaan.

A request acts as the central record connecting the user's submitted information, AI processing, geographic search, and resulting locations.

A Search Request may contain one or more inputs and may produce one or more potential location results.

4.1.2 Input

The **Input** entity stores information about the data submitted for a search request.

An input may be one of the supported types:

- ❖ Text
- ❖ Voice
- ❖ Image

Each input is associated with a particular Search Request.

4.1.3 AI Analysis

The **AI Analysis** entity represents the structured information generated after the submitted input has been processed using the Groq API.

The analysis may contain extracted:

- Landmark names
- Place names
- Descriptive clues
- Geographic clues
- Contextual information
- Other location-related information

The AI analysis is associated with the Search Request that generated it.

4.1.4 Location

The **Location** entity represents a real-world geographic place that can potentially match the information provided by a user.

A Location may contain:

- ✓ Location name
- ✓ Location type
- ✓ Description
- ✓ Latitude
- ✓ Longitude
- ✓ Geographic geometry
- ✓ OpenStreetMap reference information

The geographic information stored for a location will support spatial operations through PostGIS.

4.1.5 Search Result

The **Search Result** entity represents the relationship between a Search Request and a potentially relevant Location.

A single search may produce multiple potential locations. Each result therefore records which location was returned for a particular request.

Where applicable, the result may also contain a relevance or matching value that can be used to distinguish stronger potential matches from weaker ones.

4.2 Data Dictionary

The following data dictionary defines the principal attributes required by the Nishaan system.

4.2.1 Search Request

Attribute	Data Type	Length	Format / Allowed Values	Description
request_id	UUID	36	Unique identifier	Uniquely identifies a search request.
created_at	Timestamp	-	Date and time	Records when the request was created.
status	VARCHAR	30	pending, processing, completed, failed	Represents the current processing state of the request.

4.2.2 Input

Attribute	Data Type	Length	Format / Allowed Values	Description
input_id	UUID	36	Unique identifier	Uniquely identifies an input.
request_id	UUID	36	Existing request ID	Identifies the Search Request associated with the input.
input_type	VARCHAR	20	text, voice, image	Defines the type of input submitted by the user.
input_data	TEXT / File Reference	Variable	According to input type	Stores or references the submitted input.
created_at	Timestamp	-	Date and time	Records when the input was submitted.

4.2.3 AI Analysis

Attribute	Data Type	Length	Format / Allowed Values	Description
analysis_id	UUID	36	Unique identifier	Uniquely identifies an AI analysis.
request_id	UUID	36	Existing request ID	Identifies the Search Request being analyzed.
extracted_data	JSONB	Variable	Structured JSON	Stores location-related information extracted by the AI component.
created_at	Timestamp	-	Date and time	Records when the analysis was generated.

The use of **JSONB** allows the AI analysis to contain structured information without requiring a separate database column for every possible type of location clue.

4.2.4 Location

Attribute	Data Type	Length	Format / Allowed Values	Description
location_id	UUID	36	Unique identifier	Uniquely identifies a geographic location.
name	VARCHAR	255	Text	Name of the location where available.
type	VARCHAR	100	Text	Defines the type or category of the location.
description	TEXT	Variable	Text	Provides available descriptive information about the location.
latitude	DECIMAL	-	Geographic coordinate	Stores the latitude of the location.
longitude	DECIMAL	-	Geographic coordinate	Stores the longitude of the location.
geometry	GEOMETRY	-	PostGIS geometry	Stores geographic geometry used for spatial operations.
osm_reference	VARCHAR	100	OSM identifier	Stores the relevant OpenStreetMap reference where available.

4.2.5 Search Result

Attribute	Data Type	Length	Format / Allowed Values	Description
result_id	UUID	36	Unique identifier	Uniquely identifies a search result.
request_id	UUID	36	Existing request ID	Identifies the Search Request that generated the result.
location_id	UUID	36	Existing location ID	Identifies the geographic location associated with the result.
relevance_score	DECIMAL	-	Numeric value	Represents the relevance of the location to the submitted request where a scoring mechanism is implemented.
result_rank	INTEGER	-	Positive integer	Represents the order in which potential matches are presented.

4.2.6 Geospatial Data Requirements

Because Nishaan is a geospatial application, geographic information must be stored and processed accurately.

The system shall support:

- Latitude and longitude coordinates.
- Geographic point data.
- Spatial relationships between locations.
- Distance-based searches.
- Proximity-based searches.
- Geographic filtering.
- Geographic geometry through PostGIS.

The geometry attribute will allow PostGIS to perform spatial operations without requiring the application to perform geographic calculations manually.

For example, when a geographic point is available, the system may use PostGIS to determine which stored locations are within a particular distance of that point.

The system should maintain a consistent coordinate reference system for geographic data to prevent inaccurate spatial calculations.

4.3 Reports

Nishaan is primarily an interactive web application rather than a traditional reporting system. Therefore, Release 1.0 does not require complex administrative reports.

However, the system will provide **user-facing result information** generated from the search process.

4.3.1 Location Search Result

The location search result will provide the user with the potential locations identified by Nishaan.

It may contain:

Information	Description
Location Name	Name of the potential location where available
Location Type	Category or type of location
Description	Available contextual information
Latitude	Geographic latitude
Longitude	Geographic longitude
Relevance	Relative relevance of the result where available
Map Marker	Visual representation of the location on the map

The results will be presented through the Nishaan interface and interactive Leaflet map.

4.3.2 Search Processing Status

The system may provide processing status information such as:

- Request received
- Analyzing input
- Searching geographic data
- Results found
- No suitable match found
- Processing failed

This information allows the user to understand the current state of their request.

4.3.3 Error and No-Match Information

When the system cannot identify a suitable location, Nishaan will provide a user-readable response explaining that no sufficiently relevant result was found.

Where appropriate, the system will allow the user to provide additional information and perform another search.

4.4 Data Validation and Integrity

The system shall maintain the accuracy and integrity of stored data.

The following rules shall apply:

1. Every Search Request shall have a unique identifier.
2. Every Input shall be associated with a valid Search Request.
3. Every AI Analysis shall be associated with a valid Search Request.
4. Every Search Result shall reference a valid Location.
5. Geographic coordinates shall contain valid latitude and longitude values.
6. Geographic geometry shall contain valid spatial information before being used for spatial queries.
7. Required identifiers shall not be null.
8. Search status values shall be restricted to supported states.
9. Invalid geographic coordinates shall not be used for map visualization.
10. AI-generated information shall be validated before being used in geographic search operations.
11. Database relationships shall maintain referential integrity.
12. Spatial data shall use a consistent coordinate reference system.

4.5 Data Security and Privacy

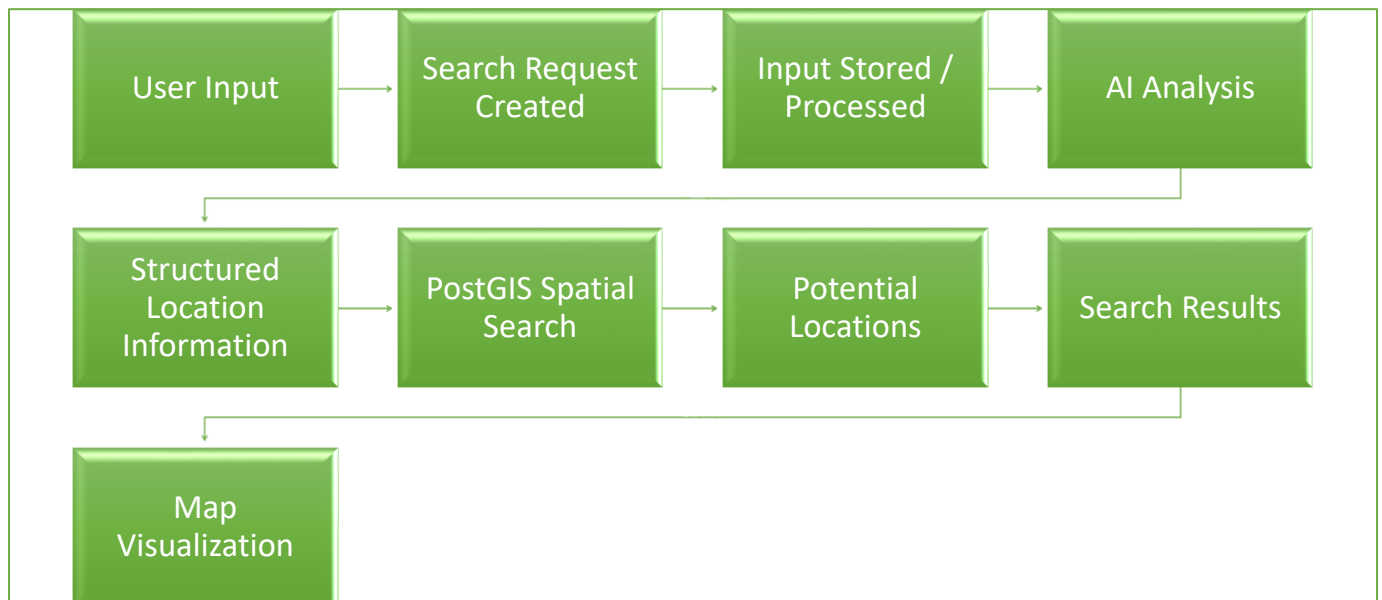
Nishaan shall apply appropriate measures to protect information processed by the system.

- ❖ API credentials shall not be stored directly in the database or exposed through the frontend.

- ❖ Sensitive configuration values shall be maintained through environment variables.
- ❖ User-provided inputs shall only be retained when required by the system's defined functionality.
- ❖ Uploaded images and voice data shall not be retained unnecessarily.
- ❖ The system shall avoid exposing internal errors, database information, API credentials, or other implementation details to users.
- ❖ Access to the database shall be restricted to authorized backend services.
- ❖ Data transmitted between the client and server shall use secure communication where supported by the deployment environment.

4.6 Data Lifecycle

The general lifecycle of Nishaan data is:



Once the search has been completed, the system may retain only the information required by the final implementation and project requirements.

5. External Interface Requirements

This section defines how Nishaan interacts with its users, software components, external services, and communication infrastructure. Since Nishaan is a web-based system that combines AI processing with geographic services, its external interfaces include the user interface, backend APIs, AI services, database services, mapping services, and communication protocols.

5.1 User Interfaces

Nishaan will provide a responsive and user-friendly web interface through which users can submit location-related information and view the resulting geographic information.

The interface will be developed using Next.js and Tailwind CSS.

5.1.1 Main Interface

The main interface will provide users with a simple entry point for location identification.

It will provide:

- A text input area for describing a location.
- A voice input option.
- An image upload option.
- A submission/action control for starting the analysis.
- Processing status information.
- An area for displaying identified results.
- An interactive map for visualizing locations.

The interface should make the three input methods clearly distinguishable so users can select the method that best suits their situation.

5.1.2 Text Input Interface

The text input interface shall allow users to enter natural-language descriptions of locations.

For example, a user may describe:

"I am looking for a mosque near a large market and a main road."

The system shall accept the description and submit it for AI-based analysis.

The interface shall:

- Provide a clearly identifiable text input area.
- Allow users to enter descriptive information.
- Prevent empty submissions where appropriate.
- Provide feedback while the request is being processed.
- Display an appropriate message when the input cannot be processed.

5.1.3 Voice Input Interface

The voice input interface shall allow users to describe a location verbally in **Urdu**.

The interface shall:

1. Provide a clearly identifiable voice-input control.
2. Request microphone permission when required.
3. Allow the user to speak their location description in Urdu.

4. Indicate when voice input is being captured.
5. Process the recorded Urdu speech through the voice-processing pipeline.
6. Pass the resulting information to the AI processing component.
7. Provide appropriate feedback if the voice input cannot be processed.

The exact voice-processing implementation will depend on the tools and services provided during the project training.

5.1.4 Image Input Interface

The image input interface shall allow users to provide a photograph or other supported image related to the location they are attempting to identify.

The interface shall:

- Provide an image upload control.
- Allow users to select a supported image.
- Validate the selected image before submission.
- Provide feedback during image processing.
- Display an appropriate message when an unsupported or invalid image is selected.

5.1.5 Processing Interface

Because Nishaan requires AI and geographic processing, the interface shall communicate the processing state to the user.

Possible states include:

- **Request received**
- **Analyzing input**
- **Searching locations**
- **Preparing results**
- **Results found**
- **No suitable match found**
- **Processing failed**

The system should prevent the user from mistakenly assuming that processing has completed while the backend is still working.

5.1.6 Results Interface

After processing is complete, the system shall present potential location results to the user.

The results interface may contain:

- Location name.

- Location type.
- Available contextual information.
- Geographic coordinates where appropriate.
- Relevance or ranking information where available.
- Interactive map markers.

The interface shall clearly distinguish between potential matches and confirmed information.

5.1.7 Map Interface

The map interface will use Leaflet with geographic map data provided through the selected OpenStreetMap integration.

The interface shall provide:

- ✓ Interactive map navigation.
- ✓ Zoom controls.
- ✓ Location markers.
- ✓ Marker selection.
- ✓ Location information associated with selected markers.
- ✓ Automatic map positioning based on returned geographic results where appropriate.

The map should remain usable on both desktop and mobile screen sizes.



Fig1: 5.1.7 Nishaan Logo

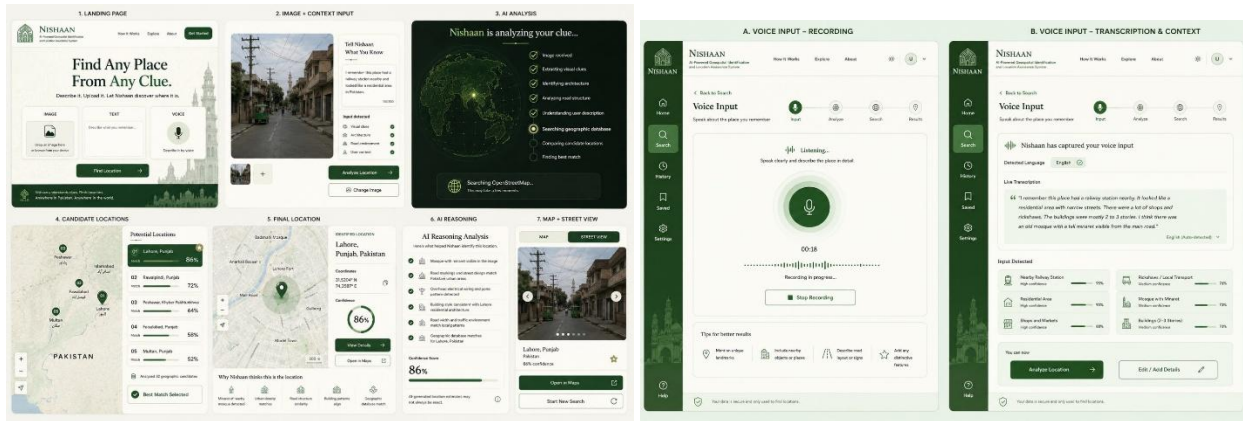


Fig2: 5.1.7 Wireframes

5.2 Software Interfaces

Nishaan will communicate with several software components and external services.

5.2.1 Frontend to Backend Interface

The Next.js frontend will communicate with the FastAPI backend through HTTP-based API requests.

The frontend will send information such as:

- Text input.
- Voice-related input.
- Image input.
- Search requests.

The backend will return structured responses containing:

- Processing status.
- AI interpretation results.
- Geographic search results.
- Location information.
- Error information where applicable.

5.2.2 FastAPI Backend Interface

The FastAPI backend will act as the central integration layer between the frontend, AI service, database, and geographic services.

The backend will be responsible for:

- Receiving client requests.

- Validating incoming data.
- Managing AI processing requests.
- Communicating with the Groq API.
- Processing structured AI output.
- Performing geographic searches.
- Communicating with PostgreSQL/PostGIS.
- Returning results to the frontend.

5.2.3 Groq API Interface

The Nishaan backend will communicate with the Groq API to perform AI-powered processing.

The interface will be used to process supported user inputs and extract meaningful location-related information.

The backend shall:

- Send appropriate input data to the AI service.
- Provide appropriate instructions for location-related analysis.
- Receive the AI response.
- Validate and process the response.
- Convert relevant information into a structured format.
- Prevent invalid AI output from being directly used for geographic searching.

The Groq API credentials shall be stored securely through environment configuration and shall not be exposed to the frontend.

5.2.4 PostgreSQL Interface

The FastAPI backend will communicate with the PostgreSQL database to store and retrieve Nishaan data.

Database operations may include:

- Creating search requests.
- Storing submitted input information where required.
- Storing AI analysis results.
- Retrieving geographic locations.
- Storing search results.
- Retrieving location details.

The backend will be responsible for validating data before performing database operations.

5.2.5 PostGIS Interface

PostGIS will extend PostgreSQL with geographic and spatial capabilities.

The backend will use PostGIS for operations such as:

- Storing geographic coordinates.
- Performing proximity searches.
- Finding locations within a geographic area.
- Comparing geographic positions.
- Retrieving geographically relevant locations.

PostGIS will provide the spatial search layer between the structured AI output and the geographic results presented to the user.

5.2.6 OpenStreetMap Interface

Nishaan will use OpenStreetMap as its geographic data source where applicable.

The exact mechanism used to retrieve or integrate OpenStreetMap data will depend on the approved service and implementation approach provided during the project training.

The system may use geographic information such as:

- ❖ Place names.
- ❖ Geographic coordinates.
- ❖ Landmark information.
- ❖ Location categories.
- ❖ Other available geographic attributes.

5.2.7 Leaflet Interface

The frontend will use Leaflet to visualize geographic information.

The backend will provide geographic results to the frontend, including coordinates and relevant location information.

Leaflet will then:

1. Load the map.
2. Position the map appropriately.
3. Display location markers.
4. Allow users to interact with markers.
5. Display available location information.

5.2.8 Environment Configuration Interface

Sensitive configuration values will be managed using environment variables.

Examples include:

- ❖ Groq API key.
- ❖ Database connection string.
- ❖ Deployment-specific configuration.
- ❖ Other sensitive credentials.

These values shall not be hard-coded into frontend or backend source files.

5.3 Hardware Interfaces

Nishaan does not require dedicated hardware for normal operation.

The system will operate using standard computing devices capable of running a modern web browser.

5.3.1 User Device

The user may access Nishaan through:

- Desktop computers.
- Laptops.
- Smartphones.
- Tablets.

5.3.2 Microphone

A microphone may be required when the user selects voice input.

The system shall rely on the user's device and browser to provide microphone access.

The system shall:

- Request permission before accessing the microphone.
- Handle permission denial appropriately.
- Provide an alternative input method if voice input cannot be used.

5.3.3 Camera

A camera may be used where image capture is supported by the browser and device.

Alternatively, users may upload an existing image from their device.

No specialized camera hardware will be required.

5.4 Communication Interfaces

Nishaan requires network communication between the frontend, backend, database, AI service, and geographic services.

5.4.1 HTTP/HTTPS

The frontend and backend will communicate through HTTP-based APIs.

For deployment, HTTPS shall be used to protect information transmitted between the user's browser and the application.

5.4.2 REST API

FastAPI will expose backend endpoints through a REST-style API architecture.

The API will support operations related to:

- Submitting user input.
- Initiating analysis.
- Retrieving processing status.
- Retrieving location results.
- Retrieving location details.
- Handling errors.

5.4.4 Secure Communication

Communication involving user information, API requests, and application data shall use secure communication mechanisms in the deployed environment.

API credentials shall never be transmitted to the frontend as part of normal client-side operations.

5.5 Interface Error Handling

Nishaan shall provide appropriate responses when an external interface fails.

Examples include:

Interface	Possible Failure	System Response
Frontend → Backend	Backend unavailable	Display service-unavailable message
Backend → Groq	AI API unavailable	Inform user that analysis cannot currently be completed
Backend → Database	Database unavailable	Display temporary processing error

PostGIS	Spatial query failure	Display search failure message
OpenStreetMap	Geographic data unavailable	Inform user that geographic information is temporarily unavailable
Leaflet	Map fails to load	Display an appropriate map error
Microphone	Permission denied	Provide alternative input method
Image Upload	Invalid file	Request another supported image

The system shall not expose internal stack traces, API credentials, database connection details, or other sensitive implementation information to users.

5.6 Interface Security Requirements

The following security requirements apply to Nishaan's external interfaces:

- **UIR1:** The system shall prevent sensitive API credentials from being exposed through the client-side application.
- **UIR2:** The system shall validate user-provided input before processing it.
- **UIR3:** The system shall use secure communication for deployed application traffic.
- **UIR4:** The system shall restrict database access to authorized backend services.
- **UIR5:** The system shall not expose internal implementation errors through user-facing responses.
- **UIR6:** The system shall request appropriate browser permissions before accessing microphone or camera capabilities.
- **UIR7:** The system shall securely manage environment variables containing sensitive configuration information.

5.7 Interface Summary

Interface	Technology	Primary Purpose
User Interface	Next.js + Tailwind CSS	User interaction
Frontend-Backend	HTTP/REST	Application communication
Backend	FastAPI + Python	System integration
AI	Groq API	AI-powered interpretation
Database	PostgreSQL	Structured data storage
Spatial Database	PostGIS	Geographic/spatial operations
Geographic Data	OpenStreetMap	Geographic information
Map	Leaflet	Interactive visualization
Deployment	Vercel + Render/Railway	Application hosting
Configuration	Environment Variables	Secure credential management

6. Quality Attributes

The quality attributes define the non-functional requirements of Nishaan. These requirements describe the expected quality, performance, security, reliability, usability, scalability, and maintainability of the system.

Since Nishaan depends on artificial intelligence, geographic data, external APIs, and spatial processing, these quality requirements are particularly important for ensuring that the system provides a reliable and understandable experience to users.

6.1 Usability

Nishaan shall provide a simple and intuitive interface that allows users to interact with the system without requiring specialized technical knowledge.

The system shall:

- **NFR1:** Provide a clear and understandable user interface.
- **NFR2:** Allow users to easily select between text, voice, and image input.
- **NFR3:** Provide clear instructions for submitting location-related information.
- **NFR4:** Display understandable feedback while requests are being processed.
- **NFR5:** Present location results in a clear format.
- **NFR6:** Clearly distinguish potential matches from confirmed information.
- **NFR7:** Provide understandable error messages without exposing technical implementation details.
- **NFR8:** Maintain a responsive interface across supported desktop and mobile screen sizes.
- **NFR9:** Minimize the number of unnecessary steps required to perform a location search.
- **NFR10:** Provide accessible controls for map interaction, input submission, and result selection.

The interface should be designed so that a first-time user can understand the primary purpose of Nishaan without requiring external instructions.

6.2 Performance

Nishaan shall provide reasonable response times while processing user requests and performing geographic searches.

The system shall:

- **NFR11:** Respond to valid frontend requests within an acceptable period under normal operating conditions.
- **NFR12:** Process database and spatial queries efficiently.
- **NFR13:** Avoid unnecessary database queries during a single search request.
- **NFR14:** Provide processing feedback when AI analysis requires additional time.

- **NFR15:** Load map results without unnecessary delay after geographic results become available.
- **NFR16:** Process multiple potential geographic results efficiently.
- **NFR17:** Handle external API delays without causing the user interface to become unresponsive.

Because AI processing and external services can introduce variable response times, the system shall communicate the processing state rather than assuming that every request will have a fixed response time.

6.3 Security

Security is required to protect the application, its data, external service credentials, and communication between system components.

The system shall:

- **NFR18:** Protect the Groq API key from exposure to users.
- **NFR19:** Store sensitive configuration values using environment variables or secure deployment configuration.
- **NFR20:** Prevent sensitive API credentials from being included in frontend source code.
- **NFR21:** Validate user-provided input before processing.
- **NFR22:** Restrict database access to authorized backend components.
- **NFR23:** Use secure communication mechanisms for deployed application traffic.
- **NFR24:** Prevent users from accessing internal system information through error messages.
- **NFR25:** Validate AI-generated information before using it for subsequent geographic processing.
- **NFR26:** Apply appropriate controls to uploaded images and voice-related data.
- **NFR27:** Prevent unauthorized manipulation of geographic or application data.

The system shall follow the principle that the frontend must never directly expose credentials required to access protected backend or external services.

6.4 Reliability

Nishaan shall provide consistent behavior when processing valid user requests.

The system shall:

- **NFR28:** Handle invalid input without terminating the application.
- **NFR29:** Handle unavailable external services gracefully.
- **NFR30:** Prevent failed AI requests from corrupting stored application data.
- **NFR31:** Prevent invalid geographic coordinates from being displayed as valid map locations.
- **NFR32:** Maintain database integrity when processing search requests.

- **NFR33:** Provide an appropriate response when no geographic match is available.
- **NFR34:** Recover gracefully from temporary failures where retrying the operation is appropriate.

Failure of one external component should not unnecessarily expose internal system errors or cause the entire user interface to become unusable.

6.5 Availability

Nishaan should remain available to users during the intended operating period.

The system shall:

- **NFR35:** Be accessible through the deployed web application when the required hosting services are operational.
- **NFR36:** Provide meaningful feedback when an external dependency is temporarily unavailable.
- **NFR37:** Avoid presenting unavailable external services as successful operations.
- **NFR38:** Allow users to retry failed requests where appropriate.

Availability of Nishaan depends partly on external services such as the Groq API, geographic data services, hosting infrastructure, and database services.

6.6 Scalability

The system should be capable of supporting an increasing number of search requests without requiring a complete redesign.

The system shall:

- **NFR39:** Use a modular frontend and backend architecture.
- **NFR40:** Use PostgreSQL/PostGIS in a manner that supports increasing geographic data and spatial queries.
- **NFR41:** Allow backend services to be scaled independently from the frontend where deployment infrastructure permits.
- **NFR42:** Use efficient database queries and appropriate indexing for frequently searched geographic data.
- **NFR43:** Allow additional AI processing capabilities to be incorporated without restructuring the entire application.

The initial implementation is intended for the hackathon and demonstration environment, but the architecture should not prevent future expansion.

6.7 Maintainability

Nishaan shall be developed using a modular structure that allows individual components to be maintained and improved independently.

The system shall:

- **NFR44:** Maintain a clear separation between frontend, backend, AI processing, database, and map components.
- **NFR45:** Use meaningful names for source-code components and database objects.
- **NFR46:** Maintain documentation for major APIs and system components.
- **NFR47:** Keep configuration values separate from application source code.
- **NFR48:** Use version control for the project source code.
- **NFR49:** Allow individual system components to be modified without unnecessary changes to unrelated components.
- **NFR50:** Maintain consistent API response structures.
- **NFR51:** Include appropriate error handling within major system components.

The architecture should allow future developers to understand the relationship between the Next.js frontend, FastAPI backend, Groq AI layer, PostgreSQL/PostGIS database, and mapping components.

6.8 Compatibility

Nishaan shall be compatible with commonly used modern web environments.

The system shall:

- **NFR52:** Support modern versions of major web browsers.
- **NFR53:** Provide a responsive interface for desktop, tablet, and mobile screen sizes.
- **NFR54:** Support devices capable of providing microphone access for voice-based interaction.
- **NFR55:** Support standard image formats accepted by the implementation.
- **NFR56:** Maintain consistent behavior across supported browsers where the required browser capabilities are available.

6.9 Data Integrity

The integrity of geographic and application data is particularly important because inaccurate coordinates can produce incorrect results on the map.

The system shall:

- **NFR57:** Maintain unique identifiers for stored entities.
- **NFR58:** Maintain valid relationships between Search Requests, Inputs, AI Analyses, Locations, and Search Results.
- **NFR59:** Validate geographic coordinates before performing spatial operations.
- **NFR60:** Maintain a consistent coordinate reference system for geographic data.

- **NFR61:** Prevent incomplete or invalid records from being used as valid geographic results.
- **NFR62:** Preserve database referential integrity.

6.10 AI Response Quality

Because Nishaan uses AI to interpret user-provided information, the quality of AI output is an important system attribute.

The system shall:

- **NFR63:** Validate AI-generated responses before passing them to the geographic search component.
- **NFR64:** Use structured AI output where required for reliable backend processing.
- **NFR65:** Handle cases where the AI cannot determine sufficient location-related information.
- **NFR66:** Avoid presenting AI-generated assumptions as confirmed geographic facts.
- **NFR67:** Provide users with appropriate feedback when the available information is insufficient for reliable matching.

AI-generated information shall be treated as **input to the geographic matching process**, rather than as the final authority on the location.

6.11 Responsiveness

The user interface should remain usable while backend and AI operations are being performed.

The system shall:

- **NFR68:** Display a loading or processing state during long-running requests.
- **NFR69:** Prevent unnecessary duplicate submissions while a request is being processed.
- **NFR70:** Update the interface when processing is completed.
- **NFR71:** Provide an appropriate response when processing fails.
- **NFR72:** Maintain responsive map controls after search results are displayed.

6.12 Deployment and Portability

Nishaan shall be deployable using the infrastructure specified for the project.

The system shall:

- **NFR73:** Support frontend deployment through Vercel.
- **NFR74:** Support backend deployment through Render or Railway.
- **NFR75:** Store deployment-specific configuration outside the application source code.
- **NFR76:** Allow the backend and frontend to communicate through configurable API endpoints.

- **NFR77:** Allow database connection information to be configured independently of application source code.

6.13 Quality Attribute Summary

Quality Attribute	Main Requirement
Usability	Simple, intuitive interaction with text, voice, image, and map
Performance	Efficient AI, database, and spatial processing
Security	Protect API keys, data, and system interfaces
Reliability	Graceful handling of invalid input and service failures
Availability	Accessible deployment with appropriate failure feedback
Scalability	Architecture capable of supporting future growth
Maintainability	Modular and well-structured implementation
Compatibility	Support modern browsers and device types
Data Integrity	Accurate and consistent application and geographic data
AI Response Quality	Validate and appropriately handle AI-generated information
Responsiveness	Clear processing states and responsive interaction
Deployment/Portability	Support Vercel and Render/Railway deployment

7. Internationalization and Localization

Nishaan is intended to serve users in Pakistan and should therefore consider the linguistic and geographic context of its target users. The system should provide a user interface that is understandable to its intended audience while allowing location descriptions to be provided in natural language.

For Release 1.0, the system will focus on supporting the primary language defined for the application interface while allowing the AI processing layer to handle location descriptions according to the capabilities of the selected AI service.

7.1 Language Support

The system shall:

- **ILR1:** Provide a consistent language throughout the primary user interface.
- **ILR2:** Allow users to provide location descriptions using natural language.
- **ILR3:** Process supported language input through the AI processing component.
- **ILR4:** Preserve the meaning of user-provided location descriptions during processing.
- **ILR5:** Display system messages, validation messages, processing states, and errors in the supported interface language.

- **ILR6:** Avoid requiring users to know the exact English name of a location when sufficient descriptive information is provided.

The exact list of supported input languages will depend on the capabilities of the final Groq-based AI implementation and will be confirmed during development.

7.1.1 Urdu Voice Input

Nishaan shall specifically support Urdu as the primary language for voice-based location descriptions in.

The system shall:

- **ILR10:** Allow users to describe locations through spoken Urdu.
- **ILR11:** Process Urdu voice input through the supported voice-processing pipeline.
- **ILR12:** Extract meaningful location-related information from Urdu speech.
- **ILR13:** Pass the extracted information to the AI processing component for further interpretation.
- **ILR14:** Preserve relevant location names, landmarks, and contextual clues from the user's Urdu description.
- **ILR15:** Provide appropriate feedback when Urdu speech cannot be understood or processed sufficiently.

For example, a user could say:

"مجھے ایسی جگہ تلاش کرنی ہے جہاں ایک بڑی مسجد کے سامنے بازار ہے اور قریب مین روڈ بھی ہے۔"

The system should use the information contained in the description to identify relevant geographic clues rather than requiring the user to know the exact name or coordinates of the location.

7.2 Pakistani Geographic Context

Since Nishaan is designed with the Pakistani context in mind, geographic information should support locations and descriptions commonly used within Pakistan.

The system should be capable of handling:

- Pakistani cities and towns.
- Local landmarks.
- Roads and streets.
- Markets.
- Public buildings.
- Educational institutions.
- Religious places.
- Parks and recreational areas.

- Other geographic entities available through the selected geographic data source.

The system should not require users to provide geographic coordinates manually when sufficient descriptive information is available.

7.3 Local Place Names

Place names may appear in different forms depending on how users describe them.

For example, users may refer to a location using:

- An official name.
- A commonly used local name.
- A shortened name.
- A landmark-based description.
- A nearby road or area.
- A combination of multiple geographic clues.

Nishaan should pass these descriptions to the AI processing component so that relevant location-related information can be extracted before geographic matching is performed.

7.4 Geographic Conventions

The system shall use internationally recognized geographic coordinate conventions internally for geographic processing.

The geographic data used by the system shall maintain:

- Latitude.
- Longitude.
- Geographic geometry where applicable.
- A consistent coordinate reference system.

These values shall be handled consistently between:

PostGIS → FastAPI → Next.js → Leaflet

This ensures that a geographic location retrieved from the database is displayed correctly on the map.

7.5 Date and Time

Nishaan does not require extensive date and time localization for its primary location-identification functionality.

Where timestamps are stored, the system shall use a consistent backend representation.

Examples include:

- ❖ Search request creation time.
- ❖ Input submission time.
- ❖ AI analysis time.
- ❖ Processing completion time.

The presentation format may be adapted for the user's environment where required.

7.6 User Interface Localization

The interface shall use terminology that is understandable to the target users.

Examples include:

Interface Element	Purpose
Describe a Location	Introduces the primary text input
Use Voice	Allows spoken input
Upload Image	Allows image-based input
Search	Starts the location analysis
Analyzing	Indicates AI processing
Finding Location	Indicates geographic search
Potential Matches	Displays possible locations
No Match Found	Indicates unsuccessful search
Try Again	Allows another search

The final wording may be adjusted during UI design and usability testing.

7.7 AI Language Processing

The AI processing component shall be capable of interpreting supported natural-language descriptions supplied by users.

The system should be able to extract relevant geographic clues from descriptions that may not follow a fixed format.

For example, a user may provide several pieces of information within one description:

"There is a large market nearby, a mosque across the road, and a main road running beside the area."

The system should process the description as contextual information rather than requiring the user to enter each clue into a separate field.

The exact languages and language combinations supported by the AI implementation shall depend on the selected Groq-compatible model and will be finalized during development.

7.8 Localization Requirements Summary

Requirement	Description
ILR1	Provide a consistent primary interface language
ILR2	Support natural-language location descriptions
ILR3	Process supported language input through AI
ILR4	Preserve the meaning of location descriptions
ILR5	Display clear localized system messages
ILR6	Support Pakistani geographic context
ILR7	Handle local place names and descriptions
ILR8	Maintain consistent geographic coordinate conventions
ILR9	Use understandable terminology throughout the interface

8. Other Requirements

This section defines additional requirements and constraints that apply to Nishaan but are not fully covered by the functional, interface, data, or quality requirements.

8.1 Legal and Regulatory Requirements

Nishaan shall use third-party services and geographic data in accordance with their applicable terms, licenses, and usage requirements.

The system shall:

- **OR1:** Use OpenStreetMap data and related services according to their applicable licensing and attribution requirements.
- **OR2:** Use the Groq API according to its applicable terms and usage limitations.
- **OR3:** Ensure that third-party API credentials are kept confidential.
- **OR4:** Avoid collecting personal information that is not required for the core functionality.
- **OR5:** Handle user-submitted images and voice data responsibly.

- **OR6:** Avoid presenting AI-generated location interpretations as guaranteed facts.

8.2 Third-Party Service Dependencies

Nishaan depends on several external technologies and services for its operation.

The primary dependencies are:

Dependency	Purpose
Groq API	AI-based processing and interpretation
OpenStreetMap	Geographic/map data
Leaflet	Interactive map visualization
PostgreSQL	Application data storage
PostGIS	Spatial and geographic operations
Vercel	Frontend deployment
Render/Railway	Backend deployment

Failure or unavailability of an external service may affect the corresponding functionality of Nishaan.

The system shall provide an appropriate user-facing response when a required external service is unavailable.

8.3 API and Usage Limitations

External services may impose request limits, processing restrictions, or other usage constraints.

The system shall:

- Avoid unnecessary repeated API requests.
- Validate input before sending it to external services.
- Handle API errors and rate-limit responses gracefully.
- Prevent accidental repeated submissions.
- Keep API credentials outside publicly accessible source code.
- Provide appropriate feedback when an external service cannot process a request.

The exact limits will depend on the services and plans used during implementation.

8.4 Data Retention

Nishaan shall follow a minimal-data approach for Release 1.0.

The system shall:

- Store only information required for the application's defined functionality.

- Avoid unnecessary permanent storage of user-submitted images and voice data.
- Avoid storing sensitive information unless required.
- Remove temporary processing data when it is no longer required by the system.
- Retain geographic information required for location matching and map visualization.

The exact retention period for temporary inputs will be finalized during implementation based on the selected services and project requirements.

8.5 Backup and Recovery

The system's persistent database data should be protected against accidental loss where the deployment environment provides suitable backup capabilities.

The system should:

- Maintain database backups where supported by the selected hosting environment.
- Allow recovery of important persistent geographic and application data.
- Avoid relying solely on temporary application storage.
- Handle temporary service failures without corrupting persistent records.

For the hackathon prototype, the level of backup and recovery will depend on the database hosting environment selected by the team.

8.6 Deployment Requirements

Nishaan shall be deployable using the technology stack specified for the project.

The deployment architecture shall support:

Frontend

Next.js → Vercel

Backend

FastAPI + Python → Render/Railway

Database

PostgreSQL + PostGIS

The deployed frontend shall communicate with the deployed backend through a configurable API endpoint.

Environment-specific configuration shall not be hard-coded into the application.

8.7 Environment Configuration

The system shall use environment variables for configuration values that differ between development and production environments.

These may include:

- Groq API key.
- Database connection URL.
- Backend API URL.
- Deployment configuration.
- Other required credentials.

Example:

```
GROQ_API_KEY=*****  
DATABASE_URL=*****  
NEXT_PUBLIC_API_URL=*****
```

Actual secret values shall never be committed to the public source-code repository.

8.8 OpenStreetMap Requirements

Where OpenStreetMap data is used, Nishaan shall follow the applicable requirements for attribution and use.

The map interface should provide appropriate attribution for the geographic data source.

The exact method of obtaining geographic data will depend on the implementation selected during development.

The system shall not assume that all geographic information is available for every location or that every user-described landmark will have an exact corresponding record.

8.9 AI Usage Requirements

Nishaan uses AI as an interpretation component rather than as the sole source of geographic truth.

Therefore:

- AI-generated information shall be treated as potentially uncertain.
- AI output shall be validated before geographic searching.
- Geographic results shall be obtained through the spatial data/search layer.
- The system shall distinguish between AI interpretation and geographic verification.
- The system shall handle cases where AI cannot extract sufficient location information.

- The system shall not claim certainty when the available evidence only supports a potential match.

This separation is an important project requirement:

User Information



Groq AI



Location Clues



PostGIS / Geographic Data



Potential Geographic Matches



Leaflet Map

8.10 Technology Constraints

Release shall be developed using the technology stack defined for the project.

Frontend

- Next.js
- Tailwind CSS

Backend

- FastAPI
- Python

Artificial Intelligence

- Groq API

Database

- PostgreSQL
- PostGIS

Geographic Data and Mapping

- OpenStreetMap
- Leaflet

Deployment

- Vercel
- Render or Railway

The team should avoid introducing additional major technologies unless required by an identified implementation need or approved during the project.

8.11 Open Source and Third-Party Components

Nishaan may use open-source libraries and third-party services as required by the selected technology stack.

The development team shall:

- Review applicable licenses for third-party libraries.
- Follow required attribution requirements.
- Keep dependencies reasonably up to date during development.
- Avoid incorporating components with incompatible licensing requirements.
- Document significant third-party dependencies where appropriate.

8.12 Project Constraints

The following constraints apply to the initial release:

1. The system is intended to be developed and demonstrated within the hackathon timeline.
2. The implementation must use the specified technology stack unless changes are required and approved.
3. The system depends on external AI and geographic services.
4. AI processing quality may vary depending on the submitted input.
5. Geographic coverage depends on the availability and quality of the underlying geographic data.
6. Internet connectivity is required for the web application and its external services.
7. External API usage may be subject to rate limits or service availability.
8. The initial release is focused on the core location-identification workflow rather than a large set of secondary features.

8.13 Future Extensibility

Although Release 1.0 will focus on the core Nishaan workflow, the architecture should allow future improvements without requiring a complete redesign.

Potential future extensions may include:

- Additional language support.
- Improved AI-based location interpretation.
- More advanced spatial matching.
- Additional geographic data sources.

- Improved ranking of potential matches.
- Additional map functionality.
- More input formats.
- Search history.
- User accounts.
- Additional geographic visualization features.

These features are **not part of the mandatory scope** unless they are later approved by the project requirements.

8.14 Other Requirements Summary

Requirement Area	Main Requirement
Legal & Regulatory	Follow applicable third-party service and data requirements
Third-Party Services	Handle external dependencies and failures
API Usage	Validate input and avoid unnecessary requests
Data Retention	Minimize unnecessary storage of user inputs
Backup & Recovery	Protect important persistent data where supported
Deployment	Support Vercel and Render/Railway
Environment Configuration	Keep credentials and deployment settings outside source code
OpenStreetMap	Follow applicable attribution and usage requirements
AI Usage	Treat AI output as interpretation, not geographic truth
Technology Constraints	Follow the specified project technology stack
Open Source	Respect applicable licenses
Project Constraints	Account for hackathon timeline and external dependencies
Extensibility	Allow future features without major architectural redesign

9. Appendices

This section provides supplementary information that supports the understanding, development, and maintenance of the Nishaan system. It contains terminology, abbreviations, assumptions, and supporting references used throughout the Software Requirements Specification.

9.1 Appendix A: Glossary

Term	Definition
------	------------

Nishaan	The proposed AI-powered location identification system.
AI	Artificial Intelligence, used by Nishaan to interpret user-provided location information.
Multimodal Input	The ability to provide information through multiple input types such as text, voice, and images.
Location Clue	A piece of information that can help identify or narrow down a geographic location.
Landmark	A recognizable geographic feature, building, place, or point of interest that can help identify an area.
Geospatial Data	Data representing geographic locations and their relationships to physical space.
Spatial Search	A search that uses geographic properties such as coordinates, distance, or proximity.
Geographic Coordinate	A numerical representation of a location, primarily expressed using latitude and longitude.
Latitude	The geographic coordinate representing a position north or south of the Equator.
Longitude	The geographic coordinate representing a position east or west of the Prime Meridian.
PostGIS	A spatial database extension for PostgreSQL that provides geographic data types and spatial operations.
OpenStreetMap (OSM)	A collaborative geographic data platform providing map and location information.
Leaflet	A JavaScript library used to create interactive maps in web applications.
Groq API	The AI service interface used by Nishaan for AI-based processing.
FastAPI	A Python framework used to implement the Nishaan backend API.
Next.js	The React-based framework used to develop the Nishaan frontend.
Tailwind CSS	A utility-first CSS framework used for styling the Nishaan interface.
API	Application Programming Interface used for communication between software components.
REST API	An HTTP-based interface used to exchange structured information between system components.
JSON	JavaScript Object Notation, a structured format used for exchanging application data.
Search Request	A single request submitted to Nishaan to identify or locate a place.
Potential Match	A geographic location that appears relevant to the user's submitted information but is not necessarily confirmed.
Relevance Score	A value representing how closely a geographic result corresponds to the available location clues.
Geometry	Spatial information representing the geographic shape or position of an entity.
Coordinate Reference System	A standardized system used to represent geographic coordinates consistently.

Environment Variable	A configuration value stored outside application source code, commonly used for credentials and deployment settings.
----------------------	--

9.2 Appendix B: Acronyms and Abbreviations

Acronym	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
DB	Database
FR	Functional Requirement
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
NFR	Non-Functional Requirement
OSM	OpenStreetMap
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
GIS	Geographic Information System
GPS	Global Positioning System
PostGIS	PostgreSQL Geographic Information System Extension

9.3 Appendix C: Assumptions

The following assumptions are made for Release 1.0:

1. Users have access to an internet-connected device with a modern web browser.
2. The user has sufficient information to provide at least one meaningful location clue.
3. The required external services are available during normal system operation.
4. The Groq API provides the required AI processing capabilities for the selected implementation.
5. Geographic information required by Nishaan is available through the selected OpenStreetMap-based data source or geographic dataset.
6. PostgreSQL and PostGIS are available and correctly configured for the backend.
7. Users provide valid and usable text, voice, or image input.
8. The browser supports the capabilities required for the selected input method.

9. The deployment environment provides the required configuration and network connectivity.
10. The project team has access to the API credentials and development resources required for implementation.
11. AI-generated information may contain uncertainty and therefore requires geographic matching before being presented as a location result.
12. Release 1.0 will focus on the core location-identification workflow rather than additional features such as permanent user accounts or social functionality.

9.4 Appendix D: Constraints

The following constraints apply to the initial implementation:

- Nishaan must use the specified project technology stack.
- The system depends on external AI and geographic services.
- Geographic results depend on the availability and quality of geographic data.
- AI results depend on the quality and completeness of user-provided information.
- External services may impose usage limits.
- Internet connectivity is required for the core system workflow.
- The project must be developed within the hackathon timeline.
- API credentials must remain confidential.
- The system should avoid unnecessary collection and retention of user data.

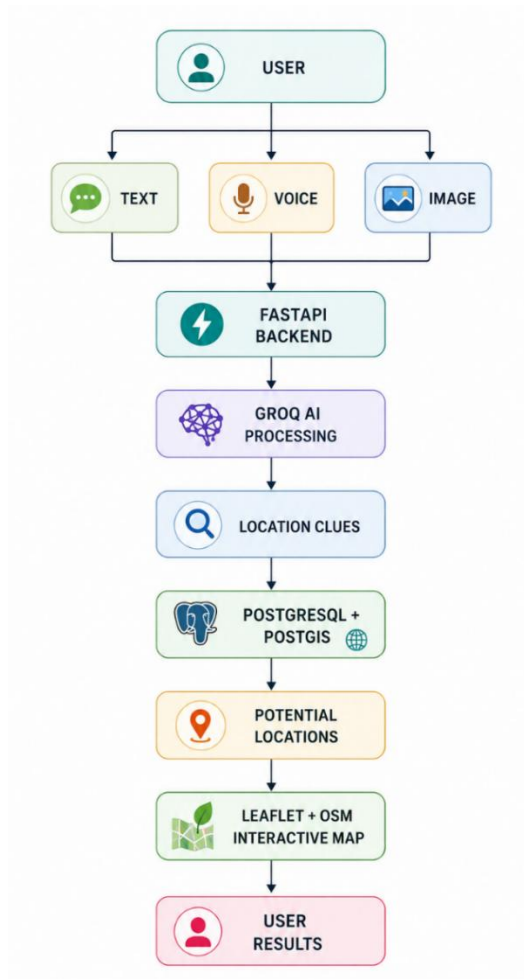
9.5 Appendix E: Technology Stack

The following technologies constitute the planned technology stack for Release 1.0:

System Component	Technology
Frontend	Next.js
UI Styling	Tailwind CSS
Backend	FastAPI + Python
AI Processing	Groq API
Database	PostgreSQL
Spatial Processing	PostGIS
Geographic Data	OpenStreetMap
Map Visualization	Leaflet
Frontend Deployment	Vercel
Backend Deployment	Render / Railway
Configuration	Environment Variables

9.6 Appendix F: Core System Flow

The overall Nishaan workflow can be summarized as follows:



9.7 Appendix G: Release 1.0 Scope

The initial release of Nishaan shall focus on the following core capabilities:

Included

- Text-based location descriptions.
- Voice-based location descriptions.
- Image-based location input.
- AI-based interpretation using Groq.
- Extraction of location-related clues.
- Geographic and spatial searching.
- PostgreSQL/PostGIS geographic data processing.
- OpenStreetMap-based geographic information.

- Interactive Leaflet map.
- Display of potential location matches.
- Location details.
- Error and no-match handling.

Not Required for Release 1.0

The following features are outside the mandatory scope unless later introduced by the project organizers:

- User registration and authentication.
- Permanent user profiles.
- Social networking functionality.
- User-to-user communication.
- Permanent search history.
- Payment functionality.
- Navigation or turn-by-turn directions.
- Complex administrative dashboards.

This keeps the initial implementation focused on the core Nishaan concept: helping a user identify a real-world location from incomplete, natural, multimodal information.

9.8 Appendix H: Requirement Traceability

The major system requirements can be traced across the SRS as follows:

Requirement Area	SRS Section
System Overview	Section 1
Overall Description	Section 2
System Features	Section 3
Data Requirements	Section 4
External Interface Requirements	Section 5
Quality Attributes	Section 6
Internationalization & Localization	Section 7
Other Requirements	Section 8
Supporting Information	Section 9